Local County School District (LCSD) Facility Use Request and Reservation Management System
Mini-Project 12: PRD Finalization, AI-Assisted Prototype & System Demonstration

Created by Julie Seibold for CPT 264-70: Systems and Procedures

LCSD Facility Use Request and Reservation Management System Product Requirements Document (PRD) for AI Implementation

Version: PRDv6 **Date:** April 2026 **AI Coding Assistant:** Windsurf
Scope: Front-end prototype with simulated backend **User Role:** Principal (Fixed)

Table of Contents

1. Executive Summary 1
2. System Overview 2
3. Assumptions and Constraints 3
4. Functional Requirements 4
5. Non-Functional Requirements 5
6. Data Model 6
7. User Interface Specifications 17
8. Program Logic & Feature Behavior 39
9. Validation & Error Handling 45
10. Data Storage & State Management 49
11. Implementation Guide for AI 51
12. Appendix 52
13. Addendum: Additional Windsurf Markdown 53

1. Executive Summary

The Local County School District (LCSD) Facility Use Request and Reservation Management System is a web-based application designed to streamline the process of requesting, reviewing, and managing after-hours and weekend facility use.

Current Challenge: LCSD lacks a centralized system, resulting in miscommunication, scheduling conflicts, and administrative inefficiencies.

Solution: A user-friendly web application that routes facility use requests through a standardized approval workflow (Requestor → Principal → Operations → Calendar).

Prototype Scope: This implementation focuses on three core features from the Principal's perspective:

1. **Submit Request** - Principals can submit facility use requests
2. **Review Request** - Principals can review and approve/deny requests for their schools
3. **Calendar View** - Principals can view and filter scheduled facility reservations

Key Constraint: This is a front-end-only prototype. Backend functionality (database, email notifications, file storage) will be simulated using JavaScript and sample data.

2. System Overview

2.1 Problem Statement

LCSD currently manages after-hours facility use through email, phone calls, and paper forms. This decentralized approach creates:

- **Communication gaps** between requestors, principals, and operations staff
- **Missed HVAC scheduling** resulting in energy waste and uncomfortable conditions
- **Lost contract renewals** due to lack of proactive expiration tracking
- **Accountability issues** when approval/denial decisions lack documentation
- **Reporting challenges** for tracking custodial, maintenance, and HVAC costs

2.2 System Purpose

The system streamlines facility reservation management by:

1. **Centralizing requests** in a single web-based platform
2. **Standardizing information collection** through structured forms
3. **Routing requests** through a defined approval workflow
4. **Maintaining a shared calendar** of facility usage
5. **Generating quotes** for paid facility use (conceptual in prototype)
6. **Tracking support needs** (HVAC, custodial, maintenance)
7. **Retaining historical records** for accountability and analysis

2.3 Key Stakeholders

For this prototype, the user is always a Principal with the following responsibilities:

- **Submit requests** for facility use at their school
- **Review and approve/deny** requests submitted by others for their school
- **View calendar** of all scheduled events at their school
- **Access reference documents** for facility use policies
- **Review archived requests** for historical reference

Other stakeholders (conceptual, not implemented as separate roles in the prototype):

- Public Requestors
- LCSD Employee Requestors
- Operations Department Staff
- Energy Manager

2.4 System Scope

In Scope for Prototype:

- Submit facility use requests (one-time and recurring)
- Review pending requests with approve/deny functionality
- View and filter calendar of scheduled events
- Access reference documents
- View archived requests
- Role-based interface (Principal view only)
- Visual status differentiation (Pending vs Approved)

Out of Scope for Prototype:

- Quote generation workflow (button present but disabled)
- Report generation (button present but disabled)
- Actual email notifications (simulated with console logs)
- Real file uploads (filename captured, file not stored)
- Backend database (data simulated in JavaScript)
- User authentication/login
- Multi-role switching

3. Assumptions and Constraints

3.1 Technology Stack

Required Technologies:

- HTML5
- CSS3 (Flexbox/Grid for layouts)
- JavaScript (ES6+, vanilla JS)

Optional Enhancements:

- Bootstrap or Tailwind CSS for styling
- Font Awesome for icons
- A lightweight calendar library (e.g., FullCalendar.js) for calendar display

Prohibited:

- Backend frameworks (Node.js, Express, Python, etc.)
- Database systems (PostgreSQL, MongoDB, MySQL, etc.)
- React, Vue, Angular, or similar frameworks
- Authentication systems
- Actual file storage or email services

3.2 Design Constraints

Course-Imposed Constraints:

- **Front-end only:** No backend server or API implementation
- **No deployment:** Runs locally in browser only
- **No database:** Data stored in JavaScript variables/arrays
- **Limited features:** Three core features only (submit, review, calendar)
- **Single role:** Principal perspective only (no role switching)

Design Goals:

1. **Accessibility:** ADA-compliant, visual status differentiation using color + icons
2. **Consistency:** Unified layout and navigation across all screens
3. **Simplicity:** Minimal visual clutter, clear action paths
4. **Usability:** All actions within 3 clicks, intuitive workflows
5. **Error Prevention:** Dropdowns for constrained choices, validation before submission
6. **Responsive:** Works on desktop, tablet, and mobile devices

3.3 Business Rules

Data Retention:

- All reservation requests retained for 5 years from event date
- Denied requests immediately archived (removed from active calendar)

Approval Workflow:

- All requests start with status "Pending"
- Principals can approve (→ "Approved"), deny (→ "Denied"), or request quote (→ "Quote Pending")
- Approved events appear on calendar with visual differentiation
- Denied events removed from calendar but retained in archived requests

Calendar Display:

- Principals see only events for their assigned school(s)
- Events visually differentiated by status (Pending: orange + clock icon; Approved: green + checkmark icon)

Request Types:

- **One-time:** Single event on specific date
- **Recurring:** Events repeating over date range (start date through end date)

4. Functional Requirements

4.1 Pre-Reservation Approval Support

- **FR-1.1: Request Submission**: The system shall allow Principals to submit facility reservation requests for one-time or recurring events at their school.
- **FR-1.2: Request Notification**: The system shall simulate notifying the Principal when a new request is submitted for their school (console log).
- **FR-1.3: Calendar Access**: The system shall provide a calendar view showing pending and approved reservations for the Principal's school.
- **FR-1.4: Quote Generation Interface**: The system shall display a "Create Quote" button on the review screen (disabled/grayed out in prototype).

4.2 Post-Decision Support

- **FR-2.1: Approval/Denial Notification**: The system shall simulate notifying requestors when their request is approved or denied (console log).
- **FR-2.2: Calendar Filtering**: The system shall allow filtering calendar views by month/year, school, and status.
- **FR-2.3: Denial Handling**: The system shall archive denied requests and remove them from the active calendar view.
- **FR-2.4: Status Updates**: The system shall update request status based on Principal decision (Pending → Approved/Denied/Quote Pending).

4.3 Ongoing Support

- **FR-3.1: Calendar Access**: The system shall allow Principals to view the calendar without modification rights (view-only mode).
- **FR-3.2: Historical Record Access**: The system shall provide access to archived requests for historical reference.
- **FR-3.3: Reference Document Access**: The system shall provide access to facility use policy documents.
- **FR-3.4: Reporting Interface**: The system shall display a "Generate Report" button on calendar view (disabled/grayed out in prototype).

5. Non-Functional Requirements

- **NFR-1:** The system shall be web-based and accessible via modern browsers (Chrome, Firefox, Safari, Edge)
- **NFR-2:** The system shall be responsive and function on desktop, tablet, and mobile devices
- **NFR-3:** The system event calendar shall visually differentiate between pending and approved events using both color and non-color indicators (icons)
- **NFR-4:** The system shall follow accessibility best practices (sufficient color contrast, keyboard navigation, semantic HTML)
- **NFR-5:** The system shall maintain consistent layout and navigation across all screens
- **NFR-6:** The system shall validate user input and display clear error messages for invalid data

6. Data Model

6.1 Entity Definitions

The system manages six primary entities. For the prototype, these are represented as JavaScript objects.

EVENT
Stores all facility reservation requests.
Attributes:

- EV_ID (Number): Unique identifier
- EV_Frequency (Text): "One-time" or "Long-term"
- EV_RequestorID (Number): USER_ID for requestor
- EV_ReviewerID (Number): USER_ID for last user to update EV_Status
- EV_Organization (Text): Name of organization associated with event
- EV_OneTimeDate (Date): Date for one-time reservation
- EV_LongTermStartDate (Date): Start date for long-term reservation
- EV_LongTermEndDate (Date): End date for long-term reservation
- EV_StartTime (Time): Start time for event
- EV_EndTime (Time): End time for event
- EV_SchoolID (Number): SCH_ID for event school
- EV_SpaceID (Number): SPACE_ID for event space
- EV_Attendees (Number): Expected number of attendees
- EV_Description (Text): Description of event
- EV_Custodial (Text): Requested custodial support
- EV_Maintenance (Text): Requested maintenance support

- EV_Files (File): Files uploaded by contact
- EV_Status (Text): "Pending", "Approved", "Quote Pending", "Quote Sent", "Denied"
- EV_Comments (Text): Comments added by Administrative Users

QUOTE
Stores quote details for paid facility contracts.
Attributes:
- QUOTE_ID (Number): Unique identifier
- QUOTE_EventID (Number): EVENT_ID for associated event
- QUOTE_CreatorID (Number): USER_ID for quote creator
- QUOTE_Custodial (Number): Amount quoted for custodial services
- QUOTE_Maintenance (Number): Amount quoted for maintenance services
- QUOTE_MiscFees (Number): Amount quoted for taxes, fees, etc.
- QUOTE_Hourly (Number): Amount quoted for hourly use of facility
- QUOTE_Status (Text): "Pending", "Accepted", "Denied"

REFERENCE_DOCUMENT
Stores reference documents for system users.
Attributes:
- DOC_ID (Number): Unique identifier
- DOC_Name (Text): Name of document
- DOC_Status (Text): "Current" or "Archived"
- DOC_UploaderID (Number): USER_ID of employee who uploaded

SCHOOL
Stores information about LCSD schools.
Attributes:
- SCH_ID (Number): Unique identifier
- SCH_Name (Text): Name of school
- SCH_Type (Text): "High School", "Middle School", "Elementary School", "K-12", "Special Programs"
- SCH_StreetAddress (Text): Street number and name for school
- SCH_City (Text): City name where school is located
- SCH_Zip (Text): ZIP code for school
- SCH_Status (Text): "Active" or "Inactive"
- SCH_User (Text): USER_ID for employee users assigned to school

SPACE
Stores reservable spaces within schools.
Attributes:
- SPACE_ID (Number): Unique identifier
- SPACE_SchoolID (Number): SCH_ID for space's school
- SPACE_Name (Text): Description of space (e.g., "Room 200", "Gym")
- SPACE_Capacity (Number): Maximum capacity for space
- SPACE_Status (Text): "Available" or "Unavailable"

USER
Stores information about all individuals who interact with the system.
Attributes:

- USER_ID (Number): Unique identifier
- USER_Type (Text): "Public User", "LCSD Employee", "Principal", "Operations Department Staff", "Energy Manager"
- USER_Name (Text): Name of user
- USER_Email (Text): Email for user
- USER_Phone (Text): Phone number for user
- USER_School (Text): Schools associated with user (can be NULL for system-wide users or public users)
- USER_Status (Text): "Active" or "Inactive"

6.2 Entity Relationships

Conceptual Relationships:

1. User requests Event → One-to-many (via EV_RequestorID)
2. User reviews Event → One-to-many (via EV_ReviewerID)
3. User creates Quote → One-to-many (via QUOTE_CreatorID)
4. Quote is generated for Event → One-to-one (via QUOTE_EventID)
5. School contains Space → One-to-many (via SPACE_SchoolID)
6. School hosts Event → One-to-many (via EV_SchoolID)
7. Space holds Event → One-to-many (via EV_SpaceID)
8. User is assigned to School → One-to-many (via USER_School)
9. User uploads Reference Document → One-to-many (via DOC_UploaderID)

For prototype: Relationships managed through matching IDs in JavaScript objects.

6.3 Sample Data Structure

Sample data will be defined in js/data.js as JavaScript arrays:
javascript
```javascript
// Sample current user (Principal)
const currentUser = {
  USER_ID: 1,
  USER_Name: "Dr. Anne Shirley",
  USER_Email: "ashirley@lcsd.edu",
  USER_Type: "Principal",
  USER_School: "Avonlea High School",
  USER_Phone: "(555) 234-5678",
  USER_Status: "Active"
};

// Sample schools
const schools = [
  {
    SCH_ID: 1,
    SCH_Name: "Avonlea High School",
    SCH_Type: "High School",
    SCH_StreetAddress: "123 Main Street",
```

```
    SCH_City: "Avonlea",
    SCH_Zip: "12345",
    SCH_Status: "Active",
    SCH_User: "1"
  },
  {
    SCH_ID: 2,
    SCH_Name: "Green Gables Middle School",
    SCH_Type: "Middle School",
    SCH_StreetAddress: "456 Oak Avenue",
    SCH_City: "Beaufort",
    SCH_Zip: "29902",
    SCH_Status: "Active",
    SCH_User: null
  },
{
    SCH_ID: 3,
    SCH_Name: "White Sands Elementary",
    SCH_Type: "Elementary School",
    SCH_StreetAddress: "789 Pine Road",
    SCH_City: "Beaufort",
    SCH_Zip: "29906",
    SCH_Status: "Active",
    SCH_User: null
  }
];

// Sample spaces
const spaces = [
  { SPACE_ID: 1, SPACE_SchoolID: 1, SPACE_Name: "Gymnasium", SPACE_Capacity: 500,
SPACE_Status: "Available" },
  { SPACE_ID: 2, SPACE_SchoolID: 1, SPACE_Name: "Cafeteria", SPACE_Capacity: 300,
SPACE_Status: "Available" },
  { SPACE_ID: 3, SPACE_SchoolID: 1, SPACE_Name: "Auditorium", SPACE_Capacity: 400,
SPACE_Status: "Available" },
  { SPACE_ID: 4, SPACE_SchoolID: 1, SPACE_Name: "Room 200", SPACE_Capacity: 30,
SPACE_Status: "Available" },
  { SPACE_ID: 5, SPACE_SchoolID: 2, SPACE_Name: "Gymnasium", SPACE_Capacity: 350,
SPACE_Status: "Available" },
  { SPACE_ID: 6, SPACE_SchoolID: 2, SPACE_Name: "Cafeteria", SPACE_Capacity: 250,
SPACE_Status: "Available" },
  { SPACE_ID: 7, SPACE_SchoolID: 3, SPACE_Name: "Gymnasium", SPACE_Capacity: 200,
SPACE_Status: "Available" }
];

// Sample events (5-10 records with various statuses)
const events = [
  {
```

```
  EV_ID: 1,
  EV_Frequency: "One-time",
  EV_RequestorID: 101,
  EV_ReviewerID: null,
  EV_Organization: "Youth Soccer League",
  EV_OneTimeDate: "2026-05-15",
  EV_LongTermStartDate: null,
  EV_LongTermEndDate: null,
  EV_StartTime: "09:00",
  EV_EndTime: "12:00",
  EV_SchoolID: 1,
  EV_SpaceID: 1,
  EV_Attendees: 45,
  EV_Description: "Youth soccer tournament for ages 10-12",
  EV_Custodial: "Setup and takedown of goals",
  EV_Maintenance: "None",
  EV_Files: "tournament_schedule.pdf",
  EV_Status: "Pending",
  EV_Comments: ""
},
{
  EV_ID: 2,
  EV_Frequency: "Long-term",
  EV_RequestorID: 102,
  EV_ReviewerID: 1,
  EV_Organization: "Community Band",
  EV_OneTimeDate: null,
  EV_LongTermStartDate: "2026-04-21",
  EV_LongTermEndDate: "2026-05-12",
  EV_StartTime: "18:00",
  EV_EndTime: "20:00",
  EV_SchoolID: 1,
  EV_SpaceID: 3,
  EV_Attendees: 60,
  EV_Description: "Community band practice every Tuesday evening",
  EV_Custodial: "None",
  EV_Maintenance: "Ensure sound system is functional",
  EV_Files: null,
  EV_Status: "Approved",
  EV_Comments: "Approved for summer session. Sound system will be checked weekly."
},
{
  EV_ID: 3,
  EV_Frequency: "One-time",
  EV_RequestorID: 103,
  EV_ReviewerID: null,
  EV_Organization: "LCSD Staff",
  EV_OneTimeDate: "2026-05-20",
```

```
  EV_LongTermStartDate: null,
  EV_LongTermEndDate: null,
  EV_StartTime: "14:00",
  EV_EndTime: "18:00",
  EV_SchoolID: 1,
  EV_SpaceID: 2,
  EV_Attendees: 100,
  EV_Description: "Staff appreciation luncheon",
  EV_Custodial: "Setup tables and chairs, cleanup after event",
  EV_Maintenance: "None",
  EV_Files: null,
  EV_Status: "Pending",
  EV_Comments: ""
},
{
  EV_ID: 4,
  EV_Frequency: "One-time",
  EV_RequestorID: 104,
  EV_ReviewerID: 1,
  EV_Organization: "Community Arts Council",
  EV_OneTimeDate: "2026-04-10",
  EV_LongTermStartDate: null,
  EV_LongTermEndDate: null,
  EV_StartTime: "19:00",
  EV_EndTime: "21:00",
  EV_SchoolID: 1,
  EV_SpaceID: 3,
  EV_Attendees: 200,
  EV_Description: "Spring concert performance",
  EV_Custodial: "Setup stage and seating",
  EV_Maintenance: "Check lighting and sound systems",
  EV_Files: "concert_program.pdf",
  EV_Status: "Denied",
  EV_Comments: "Conflicts with scheduled school event. Please contact office to reschedule."
},
{
  EV_ID: 5,
  EV_Frequency: "One-time",
  EV_RequestorID: 105,
  EV_ReviewerID: null,
  EV_Organization: "Local Basketball League",
  EV_OneTimeDate: "2026-05-22",
  EV_LongTermStartDate: null,
  EV_LongTermEndDate: null,
  EV_StartTime: "10:00",
  EV_EndTime: "14:00",
  EV_SchoolID: 1,
  EV_SpaceID: 1,
```

```
  EV_Attendees: 75,
  EV_Description: "Youth basketball tournament championship games",
  EV_Custodial: "Floor cleaning after event",
  EV_Maintenance: "Check scoreboard functionality",
  EV_Files: null,
  EV_Status: "Pending",
  EV_Comments: ""
 },
 {
  EV_ID: 6,
  EV_Frequency: "Long-term",
  EV_RequestorID: 102,
  EV_ReviewerID: 1,
  EV_Organization: "Community Band",
  EV_OneTimeDate: null,
  EV_LongTermStartDate: "2026-04-28",
  EV_LongTermEndDate: "2026-05-12",
  EV_StartTime: "18:00",
  EV_EndTime: "20:00",
  EV_SchoolID: 1,
  EV_SpaceID: 3,
  EV_Attendees: 60,
  EV_Description: "Community band practice every Tuesday evening",
  EV_Custodial: "None",
  EV_Maintenance: "Ensure sound system is functional",
  EV_Files: null,
  EV_Status: "Approved",
  EV_Comments: "Approved for summer session. Sound system will be checked weekly."
 },
 {
  EV_ID: 7,
  EV_Frequency: "Long-term",
  EV_RequestorID: 102,
  EV_ReviewerID: 1,
  EV_Organization: "Community Band",
  EV_OneTimeDate: null,
  EV_LongTermStartDate: "2026-05-05",
  EV_LongTermEndDate: "2026-05-12",
  EV_StartTime: "18:00",
  EV_EndTime: "20:00",
  EV_SchoolID: 1,
  EV_SpaceID: 3,
  EV_Attendees: 60,
  EV_Description: "Community band practice every Tuesday evening",
  EV_Custodial: "None",
  EV_Maintenance: "Ensure sound system is functional",
  EV_Files: null,
  EV_Status: "Approved",
```

```
    EV_Comments: "Approved for summer session. Sound system will be checked weekly."
  },
  {
    EV_ID: 8,
    EV_Frequency: "Long-term",
    EV_RequestorID: 102,
    EV_ReviewerID: 1,
    EV_Organization: "Community Band",
    EV_OneTimeDate: null,
    EV_LongTermStartDate: "2026-05-12",
    EV_LongTermEndDate: "2026-05-12",
    EV_StartTime: "18:00",
    EV_EndTime: "20:00",
    EV_SchoolID: 1,
    EV_SpaceID: 3,
    EV_Attendees: 60,
    EV_Description: "Community band practice every Tuesday evening",
    EV_Custodial: "None",
    EV_Maintenance: "Ensure sound system is functional",
    EV_Files: null,
    EV_Status: "Approved",
    EV_Comments: "Approved for summer session. Sound system will be checked weekly."
  },
{
    EV_ID: 9,
    EV_Frequency: "One-time",
    EV_RequestorID: 106,
    EV_ReviewerID: null,
    EV_Organization: "Science Fair Committee",
    EV_OneTimeDate: "2026-05-18",
    EV_LongTermStartDate: null,
    EV_LongTermEndDate: null,
    EV_StartTime: "08:00",
    EV_EndTime: "15:00",
    EV_SchoolID: 2,
    EV_SpaceID: 5,
    EV_Attendees: 120,
    EV_Description: "Annual middle school science fair",
    EV_Custodial: "Setup display tables, cleanup after event",
    EV_Maintenance: "None",
    EV_Files: null,
    EV_Status: "Pending",
    EV_Comments: ""
  },
  {
    EV_ID: 10,
    EV_Frequency: "One-time",
    EV_RequestorID: 107,
```

```
  EV_ReviewerID: null,
  EV_Organization: "Parent Teacher Association",
  EV_OneTimeDate: "2026-05-25",
  EV_LongTermStartDate: null,
  EV_LongTermEndDate: null,
  EV_StartTime: "18:30",
  EV_EndTime: "20:30",
  EV_SchoolID: 2,
  EV_SpaceID: 6,
  EV_Attendees: 80,
  EV_Description: "PTA monthly meeting and potluck dinner",
  EV_Custodial: "Setup tables and chairs",
  EV_Maintenance: "None",
  EV_Files: null,
  EV_Status: "Approved",
  EV_Comments: "Approved. Enjoy your event!"
},
{
  EV_ID: 11,
  EV_Frequency: "One-time",
  EV_RequestorID: 108,
  EV_ReviewerID: null,
  EV_Organization: "Elementary Book Fair",
  EV_OneTimeDate: "2026-05-12",
  EV_LongTermStartDate: null,
  EV_LongTermEndDate: null,
  EV_StartTime: "09:00",
  EV_EndTime: "16:00",
  EV_SchoolID: 3,
  EV_SpaceID: 8,
  EV_Attendees: 200,
  EV_Description: "Spring book fair for elementary students",
  EV_Custodial: "Setup book display tables",
  EV_Maintenance: "None",
  EV_Files: null,
  EV_Status: "Pending",
  EV_Comments: ""
},
{
  EV_ID: 12,
  EV_Frequency: "One-time",
  EV_RequestorID: 109,
  EV_ReviewerID: null,
  EV_Organization: "Community Reading Program",
  EV_OneTimeDate: "2026-05-30",
  EV_LongTermStartDate: null,
  EV_LongTermEndDate: null,
  EV_StartTime: "10:00",
```

```
    EV_EndTime: "12:00",
    EV_SchoolID: 3,
    EV_SpaceID: 7,
    EV_Attendees: 50,
    EV_Description: "Summer reading kickoff event for families",
    EV_Custodial: "Setup seating area",
    EV_Maintenance: "None",
    EV_Files: null,
    EV_Status: "Approved",
    EV_Comments: "Great community program. Approved."
  }
];

// Sample users (for requestor reference)
const users = [
  {
    USER_ID: 1,
    USER_Name: "Dr. Anne Shirley",
    USER_Email: "ashirley@lcsd.edu",
    USER_Type: "Principal",
    USER_Phone: "(555) 234-5678",
    USER_School: "Avonlea High School",
    USER_Status: "Active"
  },
  {
    USER_ID: 101,
    USER_Name: "Sarah Stanley",
    USER_Email: "sarah@youthsoccer.org",
    USER_Type: "Public User",
    USER_Phone: "(555) 123-4567",
    USER_School: null,
    USER_Status: "Active"
  },
  {
    USER_ID: 102,
    USER_Name: "Marilla Cuthbert",
    USER_Email: "marilla@communityband.org",
    USER_Type: "Public User",
    USER_Phone: "(555) 345-6789",
    USER_School: null,
    USER_Status: "Active"
  },
  {
    USER_ID: 103,
    USER_Name: "Felicity King",
    USER_Email: "fking@lcsd.edu",
    USER_Type: "LCSD Employee",
    USER_Phone: "(555) 456-7890",
```

```
  USER_School: null,
  USER_Status: "Active"
},
{
  USER_ID: 104,
  USER_Name: "Jasper Dale",
  USER_Email: "jasper@artscouncil.org",
  USER_Type: "Public User",
  USER_Phone: "(555) 567-8901",
  USER_School: null,
  USER_Status: "Active"
},
{
  USER_ID: 105,
  USER_Name: "Hetty King",
  USER_Email: "hetty@basketball.org",
  USER_Type: "Public User",
  USER_Phone: "(555) 678-9012",
  USER_School: null,
  USER_Status: "Active"
},
{
  USER_ID: 106,
  USER_Name: "Gilbert Blythe",
  USER_Email: "gblythe@lcsd.edu",
  USER_Type: "LCSD Employee",
  USER_Phone: "(555) 789-0123",
  USER_School: null,
  USER_Status: "Active"
},
{
  USER_ID: 107,
  USER_Name: "Sarah Stanley",
  USER_Email: "sarah.s@pta.org",
  USER_Type: "Public User",
  USER_Phone: "(555) 890-1234",
  USER_School: null,
  USER_Status: "Active"
},
{
  USER_ID: 108,
  USER_Name: "Marilla Cuthbert",
  USER_Email: "mcuthbert@bookfair.org",
  USER_Type: "Public User",
  USER_Phone: "(555) 901-2345",
  USER_School: null,
  USER_Status: "Active"
},
```

```
  {
    USER_ID: 109,
    USER_Name: "Felicity King",
    USER_Email: "felicity@reading.org",
    USER_Type: "Public User",
    USER_Phone: "(555) 012-3456",
    USER_School: null,
    USER_Status: "Active"
  }
];

// Sample reference documents
const referenceDocuments = [
  {
    DOC_ID: 1,
    DOC_Name: "Facility Use Guidelines",
    DOC_Status: "Current",
    DOC_UploaderID: 1,
    uploadDate: "2026-01-15",
    fileUrl: "assets/sample-documents/facility_guidelines.pdf"
  },
  {
    DOC_ID: 2,
    DOC_Name: "Pricing Policy",
    DOC_Status: "Current",
    DOC_UploaderID: 1,
    uploadDate: "2026-01-15",
    fileUrl: "assets/sample-documents/pricing_policy.pdf"
  },
  {
    DOC_ID: 3,
    DOC_Name: "Safety Protocols",
    DOC_Status: "Current",
    DOC_UploaderID: 1,
    uploadDate: "2026-02-01",
    fileUrl: "assets/sample-documents/safety_protocols.pdf"
  }
];

// Make data globally accessible
window.appData = {
  currentUser,
  schools,
  spaces,
  events,
  users,
  referenceDocuments
};
```

7. User Interface Specifications

7.1 Global UI Elements

Every screen includes:
Header Section:

```
+----------------------------------------+
|       [LCSD Logo/Image]            |
|  Local County School District         |
| (Centered, consistent across pages)    |
+----------------------------------------+
```

Sticky Navigation Bar:
When user scrolls down and the header logo disappears from view, a white navigation bar sticks to the top of the viewport containing:

- Home button/link (appears when logo is out of view)
- Any other relevant navigation for the current page

Implementation:

```
// Sticky nav behavior
window.addEventListener('scroll', function() {
  const header = document.querySelector('.header');
  const stickyNav = document.querySelector('.sticky-nav');
  const headerBottom = header.offsetTop + header.offsetHeight;

  if (window.pageYOffset > headerBottom) {
    stickyNav.classList.add('visible');
  } else {
    stickyNav.classList.remove('visible');
  }
});
```

CSS:

```
.sticky-nav {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  background-color: white;
  padding: 10px 20px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  z-index: 1000;
  display: none;
}

.sticky-nav.visible {
  display: block;
}
```

Page Title:
Below header, each page displays its specific title:

- "Facility Reservation Request Form"
- "Facility Reservation Pending Requests"
- "Facility Reservation Request for Review"
- "Facility Reservation Calendar"
- "Facility Reservation Reference Documents"
- "Facility Reservation Archived Requests"

7.2 Home Screen

Page Title: None (just logo and district name)
Layout:

```
+-----------------------------------------+
|       [LCSD Logo/Image]            |
|  Local County School District         |
|      Facility Reservations         |
+-----------------------------------------+
|                                 |
|  +----------------+  +----------------+ |
|  | Submit Request | |   Documents    | |
|  +----------------+  +----------------+ |
|                                 |
|  +----------------+  +----------------+ |
|  |Review Requests | |   Archived     | |
|  |            | |   Requests     | |
|  +----------------+  +----------------+ |
|                                 |
|       +----------------+          |
|       |   Calendar    |          |
|       +----------------+          |
|                                 |
+-----------------------------------------+
```

Button Grid:

- **Row 1:** Submit Request, Documents (side by side)
- **Row 2:** Review Requests, Archived Requests (side by side)
- **Row 3:** Calendar (centered, button is same width as buttons in rows above)

Button Specifications:

- Large, easily clickable
- Clear labels
- Consistent styling
- Equal size for all buttons

Interactions:

- Click "Submit Request" → Navigate to submit-request.html
- Click "Documents" → Navigate to reference-documents.html
- Click "Review Requests" → Navigate to pending-requests.html
- Click "Archived Requests" → Navigate to archived-requests.html
- Click "Calendar" → Navigate to calendar.html

Implementation:

```
<div class="home-container">
 <div class="logo-section">
  <img src="assets/lcsd-logo.png" alt="LCSD Logo" class="logo">
  <h1>Local County School District</h1>
  <h2>Facilities Reservations</h2>
 </div>

 <div class="button-grid">
  <div class="button-row">
   <button onclick="navigateTo('submit-request.html')" class="nav-button">
    Submit Request
   </button>
   <button onclick="navigateTo('reference-documents.html')" class="nav-button">
    Documents
   </button>
  </div>

  <div class="button-row">
   <button onclick="navigateTo('pending-requests.html')" class="nav-button">
    Review Requests
   </button>
   <button onclick="navigateTo('archived-requests.html')" class="nav-button">
    Archived Requests
   </button>
  </div>

  <div class="button-row centered">
   <button onclick="navigateTo('calendar.html')" class="nav-button">
    Calendar
   </button>
  </div>
 </div>
</div>

<script>
function navigateTo(page) {
 window.location.href = page;
}
</script>
```

7.3 Submit Request Screen

Page Title: "Facility Reservation Request Form"

Layout:

```
+-----------------------------------------+
|       [LCSD Logo/Image]            |
|  Local County School District         |
+-----------------------------------------+
| [Sticky Nav: Home]                  |
|                                     |
| Facility Reservation Request Form     |
|                                     |
| Request Type:                       |
| ( ) One-time    ( ) Recurring         |
|                                     |
| Name: [Dr. Anne Shirley]            |
| (auto-populated, read-only)           |
|                                     |
| Email: [ashirley@lcsd.edu]            |
| (auto-populated, read-only)           |
|                                     |
| Event Date: [__________]            |
|                                     |
| -- IF Long-term selected --           |
| End Date: [__________]              |
|                                     |
| Start Time: [__________]            |
|                                     |
| End Time: [__________]              |
|                                     |
| Location: [Dropdown - Schools]         |
|                                     |
| Space: [Dropdown - Spaces]            |
|                                     |
| Max Capacity: 500 (displays after space  |
| selection)                          |
|                                     |
| Number of Attendees: [__________]       |
| (text input with numeric pattern)      |
|                                     |
| Event Description:                  |
| [_____________________________________]      |
| (full width, matches submit button)     |
|                                     |
| Requested Custodial Support:          |
| [_____________________________________]      |
| (full width, matches submit button)     |
|                                     |
| Requested Maintenance Support:         |
```

```
| [_____]      |
| (full width, matches submit button)     |
|                                |
| File Upload: [Choose File]            |
|                                |
|         [Submit]                |
+----------------------------------------+
```

Field Specifications:

Field	Type	Required	Validation	Maps to Attribute
Request Type	Radio buttons (side by side)	Yes	Must select one	EV_Frequency
Name	Text (read-only)	Yes	N/A	Derived from currentUser.USER_Name
Email	Email (read-only)	Yes	N/A	Derived from currentUser.USER_Email
Event Date	Date picker	Yes	Must be future date	EV_OneTimeDate (one-time) or EV_LongTermStartDate (long-term)
End Date	Date picker	Conditional	Must be after Event Date	EV_LongTermEndDate
Start Time	Time picker	Yes	Valid time format	EV_StartTime
End Time	Time picker	Yes	Must be after Start Time	EV_EndTime
Location	Dropdown	Yes	Must select from list	EV_SchoolID
Space	Dropdown	Yes	Must select from list	EV_SpaceID
Max Capacity	Display only	No	N/A	Displayed from SPACE_Capacity
Number of Attendees	Text input with pattern	Yes	Numeric only, positive integer	EV_Attendees
Event Description	Textarea (full width)	Yes	Max 500 characters	EV_Description
Custodial Support	Textarea (full width)	No	Max 300 characters	EV_Custodial
Maintenance Support	Textarea (full width)	No	Max 300 characters	EV_Maintenance
File Upload	File button	No	Shows popup	EV_Files

Conditional Display Logic:
When "One-time" selected:

- Show: Event Date field
- Hide: End Date field
- Event Description label: “Event Description”

When "Recurring " selected:

- Show: Event Date field (labeled "Start Date")
- Show: End Date field
- Event Description label: "Event Description (include event frequency):"

Form Behavior:

On page load:

1. Auto-populate Name with currentUser.USER_Name
2. Auto-populate Email with currentUser.USER_Email
3. Set Name and Email fields to read-only
4. Default Request Type to "One-time"
5. Hide End Date field initially
6. Hide Max Capacity display initially
7. Populate Location dropdown with schools

When Request Type changes:

```
document.querySelectorAll('input[name="requestType"]').forEach(radio => {
  radio.addEventListener('change', function() {
    const endDateField = document.getElementById('endDateGroup');
    const eventDateLabel = document.querySelector('label[for="eventDate"]');
    const descriptionLabel = document.querySelector('label[for="description"]');

    if (this.value === 'Long-term') {
      endDateField.style.display = 'block';
      eventDateLabel.textContent = 'Start Date:';
      descriptionLabel.textContent = 'Event Description (include event frequency):';
    } else {
      endDateField.style.display = 'none';
      eventDateLabel.textContent = 'Event Date:';
      descriptionLabel.textContent = 'Event Description:';
    }
  });
});
```

When Location changes:

```
document.getElementById('location').addEventListener('change', function() {
  const selectedSchoolId = parseInt(this.value);

  if (selectedSchoolId) {
    const filteredSpaces = spaces.filter(sp => sp.SPACE_SchoolID === selectedSchoolId);

    const spaceDropdown = document.getElementById('space');
    spaceDropdown.innerHTML = '<option value="">-- Select Space --</option>';

    filteredSpaces.forEach(space => {
      const option = document.createElement('option');
      option.value = space.SPACE_ID;
      option.textContent = space.SPACE_Name;
```

```
    spaceDropdown.appendChild(option);
  });

  // Hide capacity display when location changes
  document.getElementById('capacityDisplay').style.display = 'none';
 }
});
```

When Space changes:

```
document.getElementById('space').addEventListener('change', function() {
 const selectedSpaceId = parseInt(this.value);

 if (selectedSpaceId) {
  const selectedSpace = spaces.find(sp => sp.SPACE_ID === selectedSpaceId);

  if (selectedSpace) {
   const capacityDisplay = document.getElementById('capacityDisplay');
   capacityDisplay.textContent = `Max Capacity: ${selectedSpace.SPACE_Capacity}`;
   capacityDisplay.style.display = 'block';
  }
 }
});
```

HTML Structure for Max Capacity Display:

```
<div class="form-group">
 <label for="space">Space: *</label>
 <select id="space" required>
  <option value="">-- Select Space --</option>
 </select>
 <div id="capacityDisplay" class="capacity-info" style="display: none;"></div>
 <div id="space-error" class="error-message"></div>
</div>
```

CSS for Max Capacity:

```
.capacity-info {
 margin-top: 5px;
 font-size: 0.9rem;
 color: #666;
 font-style: italic;
}
```

Number of Attendees Input:

```
<div class="form-group">
 <label for="attendees">Number of Attendees: *</label>
 <input
  type="text"
  id="attendees"
  pattern="[0-9]+"
```

```
   inputmode="numeric"
   required
  >
  <div id="attendees-error" class="error-message"></div>
</div>
```

Textarea Styling (full width):

```
.form-group textarea {
  width: 100%;
  max-width: 600px; /* Match submit button width */
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
  font-size: 1rem;
  resize: vertical;
  min-height: 80px;
}

.btn-submit {
  max-width: 600px;
  width: 100%;
  padding: 12px 40px;
  font-size: 1.1rem;
  background-color: #28a745;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  display: block;
  margin: 30px auto 0;
}
```

File Upload Button:

```
<div class="form-group">
  <label for="fileUpload">File Upload:</label>
  <button type="button" id="fileUploadBtn" class="btn-file-upload">
    Choose File
  </button>
  <div id="fileUpload-error" class="error-message"></div>
</div>

<script>
document.getElementById('fileUploadBtn').addEventListener('click', function() {
  alert('File upload has not been implemented in this prototype.');
});
</script>
```

CSS for File Upload Button:

```
.btn-file-upload {
  padding: 10px 20px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.btn-file-upload:hover {
  background-color: #0056b3;
}
```

When Submit clicked:

```
document.getElementById('submitRequestForm').addEventListener('submit', function(e) {
  e.preventDefault();

  // Validate form
  if (!validateForm()) {
    return;
  }

  // Determine request type
  const requestType = document.querySelector('input[name="requestType"]:checked').value;

  // Collect form data
  const formData = {
    EV_ID: events.length + 1,
    EV_Frequency: requestType,
    EV_RequestorID: currentUser.USER_ID,
    EV_ReviewerID: null,
    EV_Organization: document.getElementById('organization').value || currentUser.USER_Name,
    EV_SchoolID: parseInt(document.getElementById('location').value),
    EV_SpaceID: parseInt(document.getElementById('space').value),
    EV_StartTime: document.getElementById('startTime').value,
    EV_EndTime: document.getElementById('endTime').value,
    EV_Attendees: parseInt(document.getElementById('attendees').value),
    EV_Description: document.getElementById('description').value,
    EV_Custodial: document.getElementById('custodial').value,
    EV_Maintenance: document.getElementById('maintenance').value,
    EV_Files: null, // File upload not implemented
    EV_Status: "Pending",
    EV_Comments: ""
  };

  // Set date fields based on request type
  if (requestType === 'One-time') {
```

```
    formData.EV_OneTimeDate = document.getElementById('eventDate').value;
    formData.EV_LongTermStartDate = null;
    formData.EV_LongTermEndDate = null;
  } else {
    formData.EV_OneTimeDate = null;
    formData.EV_LongTermStartDate = document.getElementById('eventDate').value;
    formData.EV_LongTermEndDate = document.getElementById('endDate').value;
  }

  // Add to events array
  events.push(formData);

  // Get requestor email
  const requestorEmail = currentUser.USER_Email;

  // Simulate email notifications
  console.log('=== EMAIL SIMULATION ===');
  console.log(`To: ${requestorEmail}`);
  console.log(`Subject: Facility Request Submitted`);
  console.log(`Body: Your request has been submitted and is pending review.`);
  console.log('========================');

  // Show success message with email confirmation note
  alert('Request submitted successfully!\n\nEmail confirmation will also be sent to user.');

  // Navigate to home
  window.location.href = 'index.html';
});
```

7.4 Pending Requests Screen

Page Title: "Facility Reservation Pending Requests"

Layout:

```
+-----------------------------------------+
|       [LCSD Logo/Image]            |
|   Local County School District         |
+-----------------------------------------+
| [Sticky Nav: Home]                    |
|                                       |
| Facility Reservation Pending Requests                    |
|                                       |
| +-------------------------------------+ |
| | Requestor | Start  | End   | School | |
| |         | Date   | Date  |       | |
| | Descrip.  |       |       | Review | |
| | (preview) |       |       |       | |
| +-------------------------------------+ |
| | Sarah    | 05/15/ | N/A   | Avonlea| |
| | Stanley  | 2026   |       | HS     | |
```

```
| | Youth   |      |     | [View] | |
| | soccer...|      |     |      | |
| +-------------------------------------+ |
|                                         |
| [< Previous]  [Next >]                  |
+-----------------------------------------+
```

Table Columns:

Column	Description	Maps to Attribute
Requestor	Name from users array where USER_ID = EV_RequestorID	USER_Name
Start Date	EV_OneTimeDate or EV_LongTermStartDate	Date field
End Date	EV_LongTermEndDate or "N/A"	Date field
School	School name from schools where SCH_ID = EV_SchoolID	SCH_Name
Description	First 50 chars of EV_Description + "..."	EV_Description
Status	EV_Status badge	EV_Status
Review	View button	-

Note: No Status column as all items on this screen are "Pending"

Filtering Logic:

- Show only events where EV_Status === "Pending"
- Show only events where EV_SchoolID matches currentUser's school

Implementation Example:

```
// In pending-requests.js

function loadPendingRequests() {
  // Find current user's school ID
  const userSchool = schools.find(s => s.SCH_Name === currentUser.USER_School);
  const userSchoolId = userSchool ? userSchool.SCH_ID : null;

  // Filter pending events for user's school
  const pendingEvents = events.filter(event =>
    event.EV_Status === 'Pending' &&
    event.EV_SchoolID === userSchoolId
  );

  // Sort by start date
  pendingEvents.sort((a, b) => {
    const dateA = a.EV_OneTimeDate || a.EV_LongTermStartDate;
    const dateB = b.EV_OneTimeDate || b.EV_LongTermStartDate;
    return new Date(dateA) - new Date(dateB);
  });

  const tableBody = document.getElementById('pendingRequestsTable');
  tableBody.innerHTML = '';
```

```
    pendingEvents.forEach(event => {
    const row = document.createElement('tr');

    // Get requestor name
    const requestor = users.find(u => u.USER_ID === event.EV_RequestorID);
    const requestorName = requestor ? requestor.USER_Name : 'Unknown';

    // Get school name
    const school = schools.find(s => s.SCH_ID === event.EV_SchoolID);
    const schoolName = school ? school.SCH_Name : 'Unknown';

    // Get dates
    const startDate = event.EV_OneTimeDate || event.EV_LongTermStartDate;
    const endDate = event.EV_LongTermEndDate || 'N/A';

    // Truncate description
    const description = event.EV_Description.length > 50
      ? event.EV_Description.substring(0, 50) + '...'
      : event.EV_Description;

    row.innerHTML = `
      <td>${requestorName}</td>
      <td>${formatDate(startDate)}</td>
      <td>${endDate !== 'N/A' ? formatDate(endDate) : 'N/A'}</td>
      <td>${schoolName}</td>
      <td>${description}</td>
      <td><button onclick="viewRequest(${event.EV_ID})" class="btn-view">View</button></td>
    `;

    tableBody.appendChild(row);
  });
}

function viewRequest(eventId) {
  sessionStorage.setItem('reviewEventId', eventId);
  window.location.href = 'review-request.html';
}

function formatDate(dateString) {
  const date = new Date(dateString);
  return `${String(date.getMonth() + 1).padStart(2, '0')}/${String(date.getDate()).padStart(2,
'0')}/${date.getFullYear()}`;
}

// Load requests on page load
document.addEventListener('DOMContentLoaded', loadPendingRequests);
```

7.5 Review Request Screen

Page Title: "Facility Reservation Request for Review"

Layout (Single Column, matching Submit Request flow):

```
+-----------------------------------------+
|        [LCSD Logo/Image]            |
|   Local County School District         |
+-----------------------------------------+
| [Sticky Nav: Home] [Back to List]       |
|                                         |
| Facility Reservation Request for Review                    |
|                                         |
| REQUEST DETAILS                         |
| (Single column layout matching submit   |
| form, only using two columns where the  |
| submit form did)                        |
|                                         |
| Name: Sarah Stanley                     |
|                                         |
| Email: sarah@youthsoccer.org            |
|                                         |
| Request Type: One-time                  |
|                                         |
| Event Date: Saturday, May 15, 2026      |
|                                         |
| Start Time: 9:00 AM    End Time: 12:00 PM|
| (two columns like on submit form)       |
|                                         |
| +-------------------------------------+ |
| |  MAY 2026              < >    |  |
| |  S  M  T  W  T  F  S           |  |
| |         1  2  3  4  5          |  |
| |  6  7  8  9 10 11 12           |  |
| | 13 14 [15]●16 17 18 19         |  |
| | 20 21 22 23 24 25 26           |  |
| | 27 28 29 30 31                 |  |
| |                                |  |
| | Yellow = Pending (other requests) |  |
| | Green = Approved (other requests) |  |
| | Yellow with dot = Current request |  |
| +-------------------------------------+ |
|                                         |
| Location: Avonlea High School           |
|                                         |
| Space: Gymnasium                        |
|                                         |
| Number of Attendees: 45                 |
|                                         |
| Event Description:                      |
```

| Youth soccer tournament for ages 10-12 |
| |
| Requested Custodial Support: |

Display all event attributes:

- All event attributes shall be displayed in a single-column vertical layout matching the structure of the Submit Request form.
- Multi-column layouts are not permitted for data fields.
- The only exceptions are:
    - Action buttons row
    - Calendar month navigation row
    - Internal calendar grid structure
- Requestor Name (lookup from users array using EV_RequestorID)
- Organization (EV_Organization)
- Email (lookup from users array)
- Request Type (EV_Frequency)
- Event Date or Start Date
    - EV_OneTimeDate for one-time requests
    - EV_LongTermStartDate for recurring requests
- End Date
    - Show only for recurring requests
    - EV_LongTermEndDate
- Start Time (EV_StartTime)
- End Time (EV_EndTime)
- Calendar preview shall appear immediately below Start Time and End Time.
- Calendar preview shall default to the month of the request date.
- Left "<" button displays previous month.
- Right ">" button displays next month.
- Changing month updates:
    - Month label
    - Number of days
    - Day alignment
    - Event indicators
- Calendar preview shall show only events where:
    - EV_SchoolID matches current request school
    - EV_Status = Pending OR Approved
- Denied events shall not appear.
- Pending events use yellow indicator.
- Approved events use green indicator.
- Current request under review uses yellow highlight and dot marker.
- Calendar cells shall use fixed width and fixed height.
- All seven day columns shall remain equal width.
- Calendar row heights shall remain consistent across all weeks.
- Event indicators, markers, or text shall not expand cell width or height.
- Overflow content shall be clipped, wrapped within limits, or represented by icons/markers only.

- Changing months shall not alter calendar size.
- Text inside cells shall not resize columns or rows.
- Location (lookup school name using EV_SchoolID)
- Space (lookup space name using EV_SpaceID)
- Attendees (EV_Attendees)
- Description (EV_Description)
- Custodial Support (EV_Custodial)
- Maintenance Support (EV_Maintenance)
- Uploaded File (EV_Files)
- If EV_Files contains a filename, display clickable link text: [View Uploaded File]
- Clicking the link shall simulate reference-document behavior and open: alert("File viewing not implemented in prototype. This would open the uploaded attachment in the full system.");
- If EV_Files is null, display: No file uploaded
- Internal Comments textarea is optional for Approve and Deny.
- Internal Comments textarea is required before Create Quote can be clicked successfully.

Interactions:
When "Approve" clicked:

1. Confirm action
2. Update event:
    a. EV_Status = "Approved"
    b. EV_Comments = entered comments
    c. EV_ReviewerID = currentUser.USER_ID
3. Simulate emails
4. Show confirmation
5. Return to Pending Requests page

When "Deny" clicked:

1. Confirm action
2. Update event:
    - EV_Status = "Denied"
    - EV_Comments = [entered comments]
    - EV_ReviewerID = currentUser.USER_ID
3. Simulate email to requestor
4. Show confirmation
5. Return to Pending Requests page

When “Create Quote” clicked:

1. If comments box is blank:
    a. Show alert: “Comments are required before requesting a quote.”
2. If comments entered:
    a. Show alert: "The System would send an alert to the Operations Department Staff to generate a quote."

Implementation Example:
javascript
```
// In review-request.js

let currentEvent = null;

document.addEventListener('DOMContentLoaded', function() {
  const eventId = parseInt(sessionStorage.getItem('reviewEventId'));
  currentEvent = events.find(e => e.EV_ID === eventId);

  if (!currentEvent) {
    alert('Event not found');
    window.location.href = 'pending-requests.html';
    return;
  }

  // Get requestor info
  const requestor = users.find(u => u.USER_ID === currentEvent.EV_RequestorID);

  // Get school info
  const school = schools.find(s => s.SCH_ID === currentEvent.EV_SchoolID);

  // Get space info
  const space = spaces.find(s => s.SPACE_ID === currentEvent.EV_SpaceID);

  // Populate fields
  document.getElementById('displayName').textContent = requestor ? requestor.USER_Name :
'Unknown';
  document.getElementById('displayEmail').textContent = requestor ? requestor.USER_Email :
'Unknown';
  document.getElementById('displayOrganization').textContent = currentEvent.EV_Organization;
  document.getElementById('displayType').textContent = currentEvent.EV_Frequency;

  // Display dates based on type
  if (currentEvent.EV_Frequency === 'One-time') {
    document.getElementById('displayDate').textContent =
formatDateLong(currentEvent.EV_OneTimeDate);
  } else {
    document.getElementById('displayStartDate').textContent =
formatDateLong(currentEvent.EV_LongTermStartDate);
    document.getElementById('displayEndDate').textContent =
formatDateLong(currentEvent.EV_LongTermEndDate);
    document.getElementById('endDateSection').style.display = 'block';
  }

  document.getElementById('displayStartTime').textContent =
formatTime(currentEvent.EV_StartTime);
```

```
  document.getElementById('displayEndTime').textContent =
formatTime(currentEvent.EV_EndTime);
  document.getElementById('displayLocation').textContent = school ? school.SCH_Name :
'Unknown';
  document.getElementById('displaySpace').textContent = space ? space.SPACE_Name :
'Unknown';
  document.getElementById('displayAttendees').textContent = currentEvent.EV_Attendees;
  document.getElementById('displayDescription').textContent = currentEvent.EV_Description;
  document.getElementById('displayCustodial').textContent = currentEvent.EV_Custodial || 'None';
  document.getElementById('displayMaintenance').textContent = currentEvent.EV_Maintenance ||
'None';

if (currentEvent.EV_Files) {
document.getElementById('fileSection').style.display = 'block';

document.getElementById('fileName').innerHTML =
'<a href="#" onclick="viewUploadedFile(); return false;">View Uploaded File</a>';
} else {
document.getElementById('fileName').textContent = 'No file uploaded';
}

function viewUploadedFile() {
  alert('File viewing not implemented in prototype. This would open the uploaded attachment in the
full system.');
}

  // Build calendar preview
  const eventDate = currentEvent.EV_OneTimeDate || currentEvent.EV_LongTermStartDate;
  buildCalendarPreview(eventDate);
});

function handleApprove() {
  if (confirm('Are you sure you want to approve this request?')) {
    const comments = document.getElementById('comments').value;

    currentEvent.EV_Status = 'Approved';
    currentEvent.EV_Comments = comments;
    currentEvent.EV_ReviewerID = currentUser.USER_ID;

    // Get requestor for email
    const requestor = users.find(u => u.USER_ID === currentEvent.EV_RequestorID);

    // Simulate emails
    console.log('=== EMAIL TO REQUESTOR ===');
    console.log(`To: ${requestor.USER_Email}`);
    console.log(`Subject: Request Approved`);
    console.log(`Body: Your facility use request has been approved.`);
    console.log('==========================');
```

```
    console.log('=== EMAIL TO OPERATIONS ===');
    console.log(`Event: ${currentEvent.EV_Description}`);
    console.log(`Custodial: ${currentEvent.EV_Custodial}`);
    console.log(`Maintenance: ${currentEvent.EV_Maintenance}`);
    console.log('==========================');

    console.log('=== EMAIL TO ENERGY MANAGER ===');
    console.log(`HVAC needed for event on ${currentEvent.EV_OneTimeDate ||
currentEvent.EV_LongTermStartDate}`);
    console.log('===============================');

    alert('Request approved. Notifications sent.');
    window.location.href = 'pending-requests.html';
  }
}

function handleDeny() {
  if (confirm('Are you sure you want to deny this request? This action cannot be undone.')) {
    const comments = document.getElementById('comments').value;

    currentEvent.EV_Status = 'Denied';
    currentEvent.EV_Comments = comments;
    currentEvent.EV_ReviewerID = currentUser.USER_ID;

    const requestor = users.find(u => u.USER_ID === currentEvent.EV_RequestorID);

    console.log('=== EMAIL TO REQUESTOR ===');
    console.log(`To: ${requestor.USER_Email}`);
    console.log(`Subject: Request Denied`);
    console.log(`Body: Your facility use request has been denied.`);
    if (comments) {
      console.log(`Reason: ${comments}`);
    }
    console.log('==========================');

    alert('Request denied. Notification sent.');
    window.location.href = 'pending-requests.html';
  }
}

function handleCreateQuote() {
  alert('Quote generation feature is not available in this prototype.');
}
```

7.6 Calendar View Screen

Page Title: "Facility Reservation Calendar"

Layout:

```
+-----------------------------------------+
|        [LCSD Logo/Image]            |
|   Local County School District         |
+-----------------------------------------+
| [Home Button]                        |
|                                      |
| Facility Reservation Calendar        |
|                                      |
| +-------------------+  +---------------+ |
| |   FIXED CALENDAR      |  | Select       | |
| |   [Month grid]    |  | Filters:     | |
| +-------------------+  +---------------+ |
|                                      |
| EVENTS ON [Selected Date]:           |
| [Event cards with status badges]       |
+-----------------------------------------+
```

Filter events by:

- EV_SchoolID matches currentUser's school
- Month matches selected month
- EV_Status matches selected status filter (or "All")

Implementation Example:

javascript

```javascript
// In calendar.js

let currentMonth = new Date().getMonth();
let currentYear = new Date().getFullYear();

function applyFilters() {
  const monthYearValue = document.getElementById('monthFilter').value;
  const [year, month] = monthYearValue.split('-').map(Number);
  currentYear = year;
  currentMonth = month - 1;

  const statusFilter = document.querySelector('input[name="statusFilter"]:checked').value;

  buildCalendar(statusFilter);
}

function buildCalendar(statusFilter = 'All') {
  // Find current user's school ID
  const userSchool = schools.find(s => s.SCH_Name === currentUser.USER_School);
  const userSchoolId = userSchool ? userSchool.SCH_ID : null;
```

```
  // Filter events
  const filteredEvents = events.filter(event => {
    // School filter
    if (event.EV_SchoolID !== userSchoolId) {
      return false;
    }

    // Status filter
    if (statusFilter !== 'All' && event.EV_Status !== statusFilter) {
      return false;
    }

    // Month filter
    const eventDate = new Date(event.EV_OneTimeDate || event.EV_LongTermStartDate);
    if (eventDate.getMonth() !== currentMonth || eventDate.getFullYear() !== currentYear) {
      return false;
    }

    return true;
  });

  // Build calendar grid (implementation details...)
  // Mark days with events
  // Show status indicators inside fixed-size calendar cells
  // All day columns shall remain equal width
  // All week rows shall remain equal height
  // Text, badges, or multiple events shall not resize the grid
  // If event count exceeds display space, show compact indicators or summary count}

function displayEventsForDate(dateStr, dayEvents) {
  const eventList = document.getElementById('eventList');
  eventList.innerHTML = `<h3>Events on ${formatDateLong(dateStr)}</h3>`;

  if (dayEvents.length === 0) {
    eventList.innerHTML += '<p>No events scheduled</p>';
    return;
  }

  dayEvents.forEach(event => {
    // Get space name
    const space = spaces.find(s => s.SPACE_ID === event.EV_SpaceID);
    const spaceName = space ? space.SPACE_Name : 'Unknown';

    const card = document.createElement('div');
    card.className = `event-card status-${event.EV_Status.toLowerCase().replace(' ', '-')}`;

    card.innerHTML = `
```

```
    <div class="event-time">${formatTime(event.EV_StartTime)} -
${formatTime(event.EV_EndTime)}</div>
    <div class="event-location">${spaceName}</div>
    <div class="event-title">${event.EV_Description}</div>
    <div class="event-status ${event.EV_Status.toLowerCase()}">
      ${event.EV_Status === 'Approved' ? '✓' : '⏱'} ${event.EV_Status.toUpperCase()}
    </div>
  `;

  eventList.appendChild(card);
 });
}
```

7.7 Reference Documents Screen

Page Title: "Facility Reservation Reference Documents"
Implementation:
javascript

```
// In reference-documents.js

function loadDocuments() {
const tableBody = document.getElementById('documentsTable');
tableBody.innerHTML = '';

referenceDocuments.forEach(doc => {
const row = document.createElement('tr');
row.innerHTML = `
<td>${doc.DOC_Name}</td>
<td>${formatDate(doc.uploadDate)}</td>
<td><button onclick="viewDocument('${doc.fileUrl}')" class="btn-view">View</button></td>
`;
tableBody.appendChild(row);
});
}
function viewDocument(fileUrl) {
  window.open(fileUrl, '_blank');
}
```

7.8 Archived Requests Screen

Page Title: "Facilities Reservation Archived Requests"
Show events where:

- EV_Status === "Approved" OR EV_Status === "Denied"
- EV_SchoolID matches currentUser's school

Implementation:
javascript
```
// In archived-requests.js

function loadArchivedRequests() {
  const userSchool = schools.find(s => s.SCH_Name === currentUser.USER_School);
  const userSchoolId = userSchool ? userSchool.SCH_ID : null;

  const archivedEvents = events.filter(event =>
    (event.EV_Status === 'Approved' || event.EV_Status === 'Denied') &&
    event.EV_SchoolID === userSchoolId
  );

  // Sort by start date (most recent first)
  archivedEvents.sort((a, b) => {
    const dateA = new Date(a.EV_OneTimeDate || a.EV_LongTermStartDate);
    const dateB = new Date(b.EV_OneTimeDate || b.EV_LongTermStartDate);
    return dateB - dateA;
  });

  const tableBody = document.getElementById('archivedRequestsTable');
  tableBody.innerHTML = '';

  archivedEvents.forEach(event => {
    const requestor = users.find(u => u.USER_ID === event.EV_RequestorID);
    const school = schools.find(s => s.SCH_ID === event.EV_SchoolID);

    const startDate = event.EV_OneTimeDate || event.EV_LongTermStartDate;
    const endDate = event.EV_LongTermEndDate || 'N/A';
    const description = event.EV_Description.length > 30
      ? event.EV_Description.substring(0, 30) + '...'
      : event.EV_Description;

    const row = document.createElement('tr');
    row.innerHTML = `
      <td>${requestor ? requestor.USER_Name : 'Unknown'}</td>
      <td>${formatDate(startDate)}</td>
      <td>${endDate !== 'N/A' ? formatDate(endDate) : 'N/A'}</td>
      <td>${school ? school.SCH_Name : 'Unknown'}</td>
      <td>${description}</td>
      <td><span class="status-badge
${event.EV_Status.toLowerCase()}">${event.EV_Status}</span></td>
      <td><button onclick="viewArchived(${event.EV_ID})" class="btn-view">View</button></td>
    `;

    tableBody.appendChild(row);
  });
}
```

```
function viewArchived(eventId) {
  alert('Archived request detail view - Event ID: ' + eventId);
}
```

8. Program Logic & Feature Behavior

8.1 Feature 1: Submit Request

Triggering Action: User clicks "Submit Request" from Home
Program Flow:

```
START
  Display home screen
  User clicks "Submit Request"
  Navigate to Submit Request screen

  SYSTEM AUTO-POPULATES:
  - Name field with currentUser.USER_Name (read-only)
  - Email field with currentUser.USER_Email (read-only)

  DEFAULT SETTINGS:
  - Request Type: "One-time" selected
  - End Date field: Hidden

  INPUT: User selects Request Type (One-time or Recurring)

  IF "Recurring " selected:
    OUTPUT: Show End Date field
    Label "Event Date" becomes "Start Date"
  ELSE:
    OUTPUT: Hide End Date field
    Keep "Event Date" label

  INPUT: User selects Location (school) from dropdown


SYSTEM PROCESSING:
  - Filter spaces: WHERE SPACE_SchoolID = selected SCH_ID
  - Populate Space dropdown with filtered spaces

  INPUT: User fills out all remaining fields
  INPUT: User optionally uploads file
  INPUT: User clicks "Submit"

  VALIDATION: Check all required fields and formats

  IF validation FAILS:
    FOR EACH invalid field:
```

```
    OUTPUT: Display error message below field
    OUTPUT: Highlight field with red border
  STOP (wait for user to correct)

 IF validation SUCCEEDS:
  SYSTEM PROCESSING:
  - Generate new EV_ID = events.length + 1
  - Get selected Request Type
  - Create event object:
    EV_ID = new ID
    EV_Frequency = selected type
    EV_RequestorID = currentUser.USER_ID
    EV_ReviewerID = null
    EV_Organization = form value
    EV_SchoolID = selected school ID
    EV_SpaceID = selected space ID
    EV_StartTime = form value
    EV_EndTime = form value
    EV_Attendees = form value
    EV_Description = form value
    EV_Custodial = form value
    EV_Maintenance = form value
    EV_Files = uploaded filename or null
    EV_Status = "Pending"
    EV_Comments = ""

    IF One-time:
      EV_OneTimeDate = selected date
      EV_LongTermStartDate = null
      EV_LongTermEndDate = null
    ELSE (Long-term):
      EV_OneTimeDate = null
      EV_LongTermStartDate = selected start date
      EV_LongTermEndDate = selected end date

  - Add event object to events array

  SIMULATE ACTIONS (console.log):
  - Email to currentUser.USER_Email: "Request submitted"

  OUTPUT: Success message "Request submitted successfully!"
  Navigate to Home screen
END
```

Validation Rules:

Field	Validation	Maps to
Event Date	Required, must be future date	EV_OneTimeDate or EV_LongTermStartDate
End Date	Required if Long-term, must be > Event Date	EV_LongTermEndDate
Start Time	Required, valid time	EV_StartTime
End Time	Required, must be > Start Time	EV_EndTime
Location	Required	EV_SchoolID
Space	Required	EV_SpaceID
Attendees	Required, positive integer	EV_Attendees
Description	Required, max 500 chars	EV_Description
File	Optional, types/size constraints	EV_Files

8.2 Feature 2: Review Request

Triggering Action: User clicks "Review Requests", then selects specific request

Program Flow:

```
START
  Display home screen
  User clicks "Review Requests"
  Navigate to Pending Requests screen

  SYSTEM PROCESSING:
  - Find currentUser's school: WHERE SCH_Name = currentUser.USER_School
  - Get SCH_ID
  - Filter events array:
    WHERE EV_Status = "Pending"
    AND EV_SchoolID = user's school ID
  - Sort by EV_OneTimeDate or EV_LongTermStartDate (earliest first)

  OUTPUT: Display table of pending requests

  INPUT: User clicks "View" button on specific request

  SYSTEM PROCESSING:
  - Get EV_ID from clicked button
  - Store in sessionStorage: reviewEventId

  Navigate to Review Request screen

  SYSTEM PROCESSING:
  - Retrieve EV_ID from sessionStorage
  - Find event in events array: WHERE EV_ID = retrieved ID
  - Lookup requestor: WHERE USER_ID = event.EV_RequestorID
  - Lookup school: WHERE SCH_ID = event.EV_SchoolID
```

```
- Lookup space: WHERE SPACE_ID = event.EV_SpaceID

OUTPUT: Display all event details
OUTPUT: Display calendar preview for event month
OUTPUT: Display internal comments textarea (empty)
OUTPUT: Display three buttons: Approve, Deny, Create Quote (grayed)

INPUT: User optionally enters comments
INPUT: User clicks decision button

IF "Approve" clicked:
  CONFIRM: "Are you sure you want to approve this request?"

  IF confirmed:
    SYSTEM PROCESSING:
    - Get comments from textarea
    - Update event in events array:
      EV_Status = "Approved"
      EV_Comments = entered text
      EV_ReviewerID = currentUser.USER_ID

    SIMULATE ACTIONS (console.log):
    - Email to requestor: "Request approved"
    - Email to Operations: Event details
    - Email to Energy Manager: HVAC info

    OUTPUT: Success message "Request approved. Notifications sent."
    Navigate to Pending Requests screen

IF "Deny" clicked:
  CONFIRM: "Are you sure you want to deny this request?"

  IF confirmed:
    SYSTEM PROCESSING:
    - Get comments from textarea
    - Update event in events array:
      EV_Status = "Denied"
      EV_Comments = entered text
      EV_ReviewerID = currentUser.USER_ID

    SIMULATE ACTION (console.log):
    - Email to requestor: "Request denied" + reason

    OUTPUT: Success message "Request denied. Notification sent."
    Navigate to Pending Requests screen

IF "Create Quote" clicked:
  OUTPUT: Alert "Quote generation feature not available in prototype"
```

```
  STOP
END
```

8.3 Feature 3: Calendar View

Triggering Action: User clicks "Calendar" from Home

Program Flow:

```
START
 Display home screen
 User clicks "Calendar"
 Navigate to Calendar View screen

 SYSTEM PROCESSING:
 - Get current date
 - Find currentUser's school: WHERE SCH_Name = currentUser.USER_School
 - Get SCH_ID
 - Set initial filters:
  Month/Year = current month
  School = user's SCH_ID
  Status = "All"
 - Filter events array:
  WHERE EV_SchoolID = user's school ID
  AND event month = current month
  AND (EV_Status = "Pending" OR EV_Status = "Approved")

 OUTPUT: Display calendar grid for current month
 OUTPUT: Indicate days with events (colored dots)
 OUTPUT: Display filter controls
 OUTPUT: Display "Generate Report" button (grayed out)

 INPUT: User clicks on a date

 SYSTEM PROCESSING:
 - Get selected date
 - Filter events WHERE date matches selected date

 OUTPUT: Display event list below calendar
 FOR EACH event:
  Get space: WHERE SPACE_ID = event.EV_SpaceID
  OUTPUT: Event card showing:
   - EV_StartTime - EV_EndTime
   - SPACE_Name
   - EV_Description
   - Status badge (based on EV_Status)

 INPUT: User changes Month/Year filter

 SYSTEM PROCESSING:
 - Update currentMonth and currentYear
```

```
  - Re-filter events for new month
  - Rebuild calendar grid

  INPUT: User changes Status filter

  SYSTEM PROCESSING:
  - Re-filter events WHERE EV_Status matches selection
  - Rebuild calendar grid

  INPUT: User clicks "Generate Report"

  OUTPUT: Alert "Report generation not available in prototype"
END
```

9. Validation & Error Handling

9.1 Field-Level Validation

Validation Functions:

javascript

```javascript
// In validation.js

function validateRequired(value, fieldName) {
  if (!value || value.trim() === '') {
    return `${fieldName} is required`;
  }
  return null;
}

function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    return 'Please enter a valid email address';
  }
  return null;
}

function validateNumber(value, fieldName, mustBePositive = true) {
  const num = Number(value);
  if (isNaN(num)) {
    return `${fieldName} must be a valid number`;
  }
  if (mustBePositive && num <= 0) {
    return `${fieldName} must be a positive number`;
  }
  return null;
}
```

```
function validateDate(dateString, mustBeFuture = false) {
  const date = new Date(dateString);
  if (isNaN(date.getTime())) {
    return 'Please select a valid date';
  }
  if (mustBeFuture) {
    const today = new Date();
    today.setHours(0, 0, 0, 0);
    if (date < today) {
      return 'Date must be in the future';
    }
  }
  return null;
}

function validateDateRange(startDate, endDate) {
  const start = new Date(startDate);
  const end = new Date(endDate);
  if (end <= start) {
    return 'End date must be after start date';
  }
  return null;
}

function validateTimeRange(startTime, endTime) {
  if (startTime >= endTime) {
    return 'End time must be after start time';
  }
  return null;
}

function validateFile(file) {
  if (!file) return null;

  const allowedExtensions = ['.pdf', '.doc', '.docx', '.xls', '.xlsx'];
  const fileExtension = file.name.substring(file.name.lastIndexOf('.')).toLowerCase();

  if (!allowedExtensions.includes(fileExtension)) {
    return 'File must be PDF, Word, or Excel format';
  }

  const maxSize = 10 * 1024 * 1024; // 10MB
  if (file.size > maxSize) {
    return 'File size must not exceed 10MB';
  }

  return null;
}
```

```
function showError(fieldId, message) {
  let errorElement = document.getElementById(`${fieldId}-error`);

  if (!errorElement) {
    errorElement = document.createElement('div');
    errorElement.id = `${fieldId}-error`;
    errorElement.className = 'error-message';

    const field = document.getElementById(fieldId);
    field.parentNode.appendChild(errorElement);
  }

  errorElement.textContent = message;
  errorElement.style.display = 'block';

  const field = document.getElementById(fieldId);
  field.classList.add('error-field');
}

function clearError(fieldId) {
  const errorElement = document.getElementById(`${fieldId}-error`);
  if (errorElement) {
    errorElement.style.display = 'none';
    errorElement.textContent = '';
  }

  const field = document.getElementById(fieldId);
  if (field) {
    field.classList.remove('error-field');
  }
}

function clearAllErrors() {
  const errorMessages = document.querySelectorAll('.error-message');
  errorMessages.forEach(el => {
    el.style.display = 'none';
    el.textContent = '';
  });

  const errorFields = document.querySelectorAll('.error-field');
  errorFields.forEach(field => {
    field.classList.remove('error-field');
  });
}
```

9.2 Form-Level Validation

When user attempts to submit a form, system checks:

- All required fields are filled
- All field-level validations pass
- Logical consistency (e.g., end date after start date)

If validation fails:

- Display error summary at top of form: "Please correct the errors below"
- Highlight invalid fields with red border
- Display specific error message below each invalid field
- Scroll to first invalid field
- Do not allow form submission

If validation succeeds:

- Remove all error messages
- Allow form submission to proceed

9.3 Data Loss Prevention

Unsaved Changes Warning:
If user attempts to navigate away from a form without submitting:
javascript

```
// Trigger warning if form has unsaved changes
window.onbeforeunload = function(e) {
  if (formHasChanges) {
    e.preventDefault();
    return "You have unsaved changes. Are you sure you want to leave?";
  }
};
```

Applies to:

- Submit Request form (if user has entered data but not submitted)
- Review Request screen (if admin has entered comments but not made decision)

Does NOT apply to:

- Calendar view (filters are temporary by design)
- Read-only screens

9.4 System Error Handling

For prototype, handle:

- Missing data (e.g., sample data not loaded)
- Invalid state (e.g., user tries to review request that doesn't exist)
- Simulated network errors

Error Display:

- User-friendly message (not technical jargon)
- Suggestion for resolution
- Example: "Unable to load requests. Please refresh the page and try again."

10. Data Storage & State Management

10.1 Conceptual Backend

Database Type: Relational database (PostgreSQL)
Data Organization: Data is organized into six primary tables corresponding to the entities: Event, User, School, Space, Quote, Reference Document. Tables are related through foreign keys:

- Event references User via RequestorID and ReviewerID
- Event references School via SchoolID
- Event references Space via SpaceID
- Quote references Event via EventID
- Quote references User via CreatorID
- Space references School via SchoolID
- Reference Document references User via UploaderID

Data Access: System components access data through SQL operations:

- CREATE: INSERT statements add new records (new requests, quotes, users)
- READ: SELECT statements retrieve data for display (pending requests, calendar events, user profiles)
- UPDATE: UPDATE statements modify existing records (status changes, comment additions)
- ARCHIVE: Denied/completed requests flagged as archived rather than deleted

10.2 Data Lifecycle

Data Creation:

- New Event records created when users submit reservation requests
- New Quote records created when Operations Staff generates quotes
- New User records created when public requestors first submit (or employees added to system)
- New Reference Document records created when admins upload files

Data Modification:

- Event status changes (Pending → Approved/Denied/Quote Pending/Quote Sent)
- Event comments added during review process
- Quote amounts adjusted by Operations Staff
- User information updated (email changes, status changes to inactive)

Data Retrieval:

- Pending requests queried for review lists (filtered by school association)
- Calendar events queried for calendar display (filtered by date range, school, status)
- User profiles queried for authentication and role-based access
- Archived requests queried for historical reference

Data Deletion/Archival:

- Denied requests immediately archived (removed from active calendar, retained in database)
- All event data retained for 5 years from event date
- After 5 years, event records automatically deleted
- Users set to "Inactive" rather than deleted

10.2 State Management During User Sessions

Session State: When user accesses the system, temporary state is maintained for:

- User Identity: USER_ID, USER_Type, USER_Name, USER_School
- Current Screen: Which page user is viewing
- Form State: Data entered in forms before submission
- Filter State: Calendar filter selections (month, school, status)

State Persistence Rules:
Temporary State (exists only during session):

- Form input before submission
- Calendar filter selections
- Selected date on calendar
- Scroll position, UI preferences

If user leaves screen without saving:

- Form data is lost
- Filters reset to defaults
- No data written to database

Persistent State (written to database):

- Submitted reservation requests
- Approval/denial decisions
- Generated quotes
- Internal comments
- Status changes

State is persisted when user clicks:

- "Submit" (on request form)
- "Approve/Deny/Request Quote" (on review screen)
- "Send Quote" (on quote generation screen)

Example - Calendar Filters: User applies filters (May 2026, Lincoln High, Approved only). These selections are stored in JavaScript variables during the session. When user navigates away from calendar or logs out, filters are cleared. Next time user (or another user) accesses calendar, default filters apply (current month, user's schools, all statuses).

10.4 Prototype Data Simulation Strategy

Since no backend database will be implemented, the AI-assisted prototype will simulate data storage using hard-coded sample data and front-end state management.
Sample Data Structure: The prototype will include predefined sample records for each data type to demonstrate system functionality:

Limitations of Prototype Simulation:

1. No persistence across sessions:
    a. Data changes (new requests, status updates) exist only in browser memory
    b. Refreshing page resets all data to original sample set
    c. No ability to save changes permanently

2. No multi-user support:
    a. All users share same sample dataset
    b. No real authentication or user sessions
    c. User role simulated with simple variable or URL parameter
3. Email notifications simulated:
    a. Actual emails not sent
    b. Notifications shown as console.log messages or brief UI alerts
    c. No email delivery tracking
4. File uploads simulated:
    a. Files not actually uploaded to server
    b. Only filename stored in event record
    c. File "download" shows alert with filename or opens sample PDF
5. Limited data volume:
    a. Only 5-10 sample events (vs. thousands in real system)
    b. Limited date ranges (primarily current/upcoming months)
    c. Pagination simulated but not truly necessary with small dataset

These limitations are acceptable for demonstrating system functionality and will be clearly noted in the video demonstration.

11. Implementation Guide for AI

11.1 File Structure

```
lcsd-prototype/
 ├── index.html
 ├── submit-request.html
 ├── pending-requests.html
 ├── review-request.html
 ├── calendar.html
 ├── reference-documents.html
 ├── archived-requests.html
 ├── css/
 │   └── styles.css
 ├── js/
 │   ├── data.js
 │   ├── validation.js
 │   ├── utils.js
 │   ├── submit-request.js
 │   ├── pending-requests.js
 │   ├── review-request.js
 │   ├── calendar.js
 │   ├── reference-documents.js
 │   └── archived-requests.js
 └── assets/
     ├── lcsd-logo.png
     └── sample-documents/
```

11.2 Sample Data Implementation

Use the exact structure from section 6.3 with all original attribute names preserved.

11.3 CSS Styling Requirements

Use the CSS from the earlier version, ensuring status badges and visual differentiation use both color and icons for accessibility.

11.4 JavaScript Implementation

Key Points:

- Use original attribute names (EV_ID, USER_Name, SCH_ID, etc.) throughout
- Filter by matching IDs: event.EV_SchoolID === userSchool.SCH_ID
- Lookup related data: users.find(u => u.USER_ID === event.EV_RequestorID)
- Console.log for email simulations
- SessionStorage for passing EV_ID between screens

11.5 Testing Checklist

Test all functionality ensuring attribute names are used correctly throughout.

12. Appendix

12.1 Use Case Summaries

UC-Request 1: Submit One-Time Event Request

- Actor: Principal (submitting as requestor)
- Trigger: User clicks "Submit Request"
- Main Flow: Select one-time, fill form, validate, submit, receive confirmation
- Postcondition: Request created with Pending status, notifications sent

UC-Request 2: Submit Recurring Event Request

- Actor: Principal
- Trigger: User clicks "Submit Request"
- Main Flow: Select recurring, fill form with start/end dates, validate, submit
- Postcondition: Recurring request created with Pending status

UC-Review 1: Review and Approve Request

- Actor: Principal
- Trigger: User clicks "Review Requests" then selects specific request
- Main Flow: View details, add comments, click Approve, notifications sent
- Postcondition: Request status = Approved, appears on calendar

UC-Review 2: Review and Deny Request

- Actor: Principal
- Trigger: User reviews pending request
- Main Flow: View details, add comments/reason, click Deny
- Postcondition: Request status = Denied, archived, removed from calendar

UC-Calendar 1: View Calendar

- Actor: Principal
- Trigger: User clicks "Calendar"
- Main Flow: System displays events for user's school
- Postcondition: Calendar visible with events

UC-Calendar 2: Filter Calendar

- Actor: Principal
- Trigger: User changes filter selections
- Main Flow: Select month, status; system refreshes display
- Postcondition: Filtered calendar view

UC-Documentation 1: View Reference Documents

- Actor: Principal
- Trigger: User clicks "Documents"
- Main Flow: Select document, view in new tab
- Postcondition: Document viewed

UC-Documentation 2: View Archived Requests

- Actor: Principal
- Trigger: User clicks "Archived Requests"
- Main Flow: View list, select specific request
- Postcondition: Historical request viewed

12.2 Design Goals

1. **Accessibility:** Follow ADA guidelines, visual differentiation using color + icons
2. **Compliance:** Conform to LCSD policies
3. **Portability:** Consistent behavior across devices/browsers
4. **Reliability:** Simulate 24/7 availability (conceptually)
5. **Security:** Demonstrate role-based access controls (conceptually)
6. **Usability:** User-friendly, intuitive, all actions within 3 clicks

12.3 Glossary

Administrative User: Principal, Operations Staff, or Energy Manager with elevated permissions
Approved: Status indicating request has been approved and event is scheduled
Archived: Denied or past events retained for historical reference
Conceptual Backend: Design of how data would be stored/managed if system were fully implemented
Duration: Length of event in hours
Event: A facility reservation request (pending, approved, or denied)
LCSD: Local County School District
One-time Event: Single occurrence reservation
Pending: Status indicating request awaiting review
Principal: School administrator responsible for approving facility use
Prototype: Front-end-only demonstration of system functionality
Quote Pending: Status indicating long-term request requires pricing quote
Recurring Event: Reservation repeating over date range
Reference Document: Policy or guideline document available to all users

Requestor: Person submitting facility use request
Space: Specific reservable area within a school (gym, cafeteria, room)
Status: Current state of request (Pending, Approved, Denied, Quote Pending, Quote Sent)

13. Additional Windsurf Markdown

13.1 Post-Initial Implementation Changes: Final Tweaks Summary

The above documentation was generated with the assistance of Claude 4.5 and ChatGPT. After putting the information into Windsurf, additional direction was given to Windsurf. The information below details the 23 final tweaks implemented to enhance the LCSD Facility Use Request and Reservation Management System prototype. These improvements focus on usability, visual consistency, and functionality enhancements.

Completed Enhancements

1. Always-Visible Home Button

- **Files Modified**: All HTML pages (`index.html`, `submit-request.html`, `pending-requests.html`, `review-request.html`, `calendar.html`, `reference-documents.html`, `archived-requests.html`)
- **Changes**: Added a persistent home button in the header of all pages with hover effects
- **Impact**: Improved navigation consistency across the application

2. Organization Field Repositioning

- **Files Modified**: `submit-request.html`
- **Changes**: Moved organization field from bottom of form to position directly under email field
- **Impact**: Better form flow and logical field organization

3. Calendar Button Centering

- **Files Modified**: `css/styles.css`
- **Changes**: Updated CSS to properly center the Calendar button in its row on the home screen
- **Impact**: Improved visual balance and professional appearance

4. Interactive Calendar Days on Review Page

- **Files Modified**: `review-request.html`, `js/review-request.js`, `css/styles.css`
- **Changes**:
  - Added clickable calendar days with event indicators
  - Implemented event preview functionality showing time, space, and description
  - Added preview section styling
- **Impact**: Enhanced user experience with interactive calendar exploration

5. Archived Requests Filtering Enhancement

- **Files Modified**: `js/archived-requests.js`
- **Changes**: Updated filtering logic to show only denied events and past approved events
- **Impact**: More relevant archived request display

6. School Dropdown Filter on Calendar

- **Files Modified**: `calendar.html`, `js/calendar.js`
- **Changes**:
  - Added school dropdown filter to calendar page
  - Implemented dynamic school population from data
  - Added filtering logic for school-specific events
- **Impact**: Enhanced calendar filtering capabilities

7. School Names in Calendar Event Descriptions

- **Files Modified**: `js/calendar.js`
- **Changes**: Added school name display in event descriptions on calendar page
- **Impact**: Better event context and identification

8. Calendar Page Click Mapping Fix

- **Files Modified**: `js/calendar.js`
- **Changes**: Fixed date highlighting and selection logic to correctly map clicked dates
- **Impact**: Resolved calendar interaction issues

9. Generic School Building Logo

- **Files Modified**: All HTML pages, `css/styles.css`
- **Changes**:
  - Replaced image-based logo with CSS-drawn school building icon
  - Added pseudo-elements and styling for scalable logo
- **Impact**: Consistent branding without external image dependencies

10. Review Request Calendar Dot Positioning

- **Files Modified**: `js/data.js`
- **Changes**: Updated sample event date from May 15 to May 14 to match user observation
- **Impact**: Correct calendar dot positioning for current events

11. Archived Request Page Title Fix

- **Files Modified**: `archived-requests.html`
- **Changes**: Fixed typo from "Facilities" to "Facility" in page title
- **Impact**: Corrected page title consistency

12. Archived Request View Popup

- **Files Modified**: `js/archived-requests.js`
- **Changes**: Replaced detailed view with prototype message popup
- **Impact**: Clear communication about prototype limitations

13. Calendar Default to Principal's School

- **Files Modified**: `js/calendar.js`
- **Changes**: Updated school dropdown to default to principal's school instead of "All Schools"
- **Impact**: More relevant default calendar view

14. Quote Needed Status Option

- **Files Modified**: `calendar.html`, `js/calendar.js`, `css/styles.css`
- **Changes**:
  - Added "Quote Needed" status filter option
  - Implemented styling for quote-pending events
  - Updated filtering logic to include quote-pending status
- **Impact**: Enhanced status tracking capabilities

15. Calendar Date Clicking Bug Fix

- **Files Modified**: `js/calendar.js`
- **Changes**: Fixed date selection logic to correctly highlight and display events for clicked dates
- **Impact**: Resolved May 22 selection issue

16. Calendar Button Sizing Consistency

- **Files Modified**: `css/styles.css`
- **Changes**: Ensured calendar button has same size and shape as other navigation buttons
- **Impact**: Visual consistency across navigation elements

17. Timezone Date Issue Resolution

- **Files Modified**: `js/calendar.js`, `js/review-request.js`
- **Changes**:
  - Fixed `formatDateLong` function to avoid timezone issues
  - Added explicit date formatting with timezone-safe approach
- **Impact**: Correct date display regardless of timezone

18. Home Page Calendar Button Size Fix

- **Files Modified**: `css/styles.css`
- **Changes**: Updated CSS to ensure calendar button matches other button sizes
- **Impact**: Consistent button sizing on home page

19. Calendar Button Width Matching

- **Files Modified**: `css/styles.css`
- **Changes**: Adjusted grid layout to make Calendar button same width as Documents button
- **Impact**: Perfect visual alignment and consistency

20. Apply Filters Button Removal

- **Files Modified**: calendar.html
- **Changes**: Removed the "Apply Filters" button while keeping the "Generate Report" button
- **Impact**: Filters now apply automatically when changed, improving user experience

21. Calendar Button Width Adjustment

- **Files Modified**: css/styles.css
- **Changes**:
  - Updated centered button layout to make Calendar button same width as Submit Request button
- **Impact**: Perfect visual alignment and consistency with other navigation buttons

22. Required Comments for Denials

- **Files Modified**: review-request.html, js/review-request.js, css/styles.css
- **Changes**:
  - Added validation to require comments before allowing request denial
  - Added visual indicators (red asterisk and explanatory note) for required field
  - Implemented focus behavior when validation fails
- **Impact**: Ensures proper documentation for denial decisions and improves data quality

23. Generate Report Functionality

- **Files Modified**: calendar.html, js/calendar.js
- **Changes**:
  - Enabled the Generate Report button (removed disabled attribute)
  - Added generateReport() function with informative prototype message
  - Message explains potential for after-hours HVAC usage and support data reporting
- **Impact**: Provides clear communication about future reporting capabilities

Technical Implementation Details

CSS Enhancements

- Added responsive button grid layouts
- Implemented CSS-based school building logo using pseudo-elements
- Enhanced event status styling with new color schemes
- Improved hover states and transitions

JavaScript Improvements

- Enhanced date formatting to handle timezone issues
- Improved calendar interaction logic
- Added dynamic filtering capabilities
- Implemented event preview functionality

Data Structure Updates

- Updated sample event dates for consistency
- Maintained existing data structure while enhancing functionality

User Experience Improvements

Navigation

- Persistent home button across all pages
- Consistent button sizing and styling
- Improved visual hierarchy

Calendar Functionality

- Interactive calendar days with event previews
- Enhanced filtering options (school + status)
- Correct date selection and highlighting
- Timezone-safe date display

Visual Consistency

- Uniform button dimensions and styling
- Professional CSS-based branding
- Consistent color schemes and status indicators

Files Modified Summary

HTML Files (7)

- `index.html` - Home page with navigation updates
- `submit-request.html` - Form field reorganization
- `pending-requests.html` - Header updates
- `review-request.html` - Calendar preview enhancements and required comment indicators
- `calendar.html` - School dropdown, status filters, and enabled Generate Report button
- `reference-documents.html` - Header updates
- `archived-requests.html` - Title fix and header updates

CSS Files (1)

- `css/styles.css` - Comprehensive styling updates including required field indicators and button sizing

JavaScript Files (4)

- `js/calendar.js` - Calendar functionality, filtering, and generateReport function
- `js/review-request.js` - Event preview, date formatting, and denial validation
- `js/archived-requests.js` - Filtering and popup functionality
- `js/data.js` - Sample data updates for consistency

Conclusion

These 23 enhancements significantly improve the LCSD Facility Management System prototype's usability, visual consistency, and functionality. The system now provides a more professional and intuitive user experience with enhanced calendar features, improved navigation, consistent visual design, proper validation controls, and clear communication about prototype capabilities.

All requested features have been implemented and tested, ensuring the prototype meets the specified requirements while maintaining responsive design and accessibility standards.

END OF PRODUCT REQUIREMENTS DOCUMENT

Additional documentation at:
https://github.com/data-hooper/systemsandprocedures/tree/main

High-Level Project Schedule

Phase	Topic	Projected Completion Date	Actual Completion Date
Planning			
	System Vision & Problem Statement	1/25/26	1/21/26
	Project Selection & Management	2/1/26	1/28/26
Analysis			
	Functional & Non-Functional Requirements Specification	2/8/26	2/4/26
	Use Case Specification & User Interaction Analysis	2/15/26	2/13/26
	System Process Models & Workflow Analysis	2/22/26	2/21/26
	Conceptual Data Model & Data Dictionary	3/1/26	3/1/26
Design			
	System Design Strategy & Transition Plan	3/8/26	3/6/26
	System Architecture Diagram & Component Design	3/15/26	3/13/26
	User Interface Wireframes & Interaction Design	3/22/26	3/14/26
	Program Logic & Front-End Behavior Design	4/5/26	3/26/26
	Conceptual Data Storage & State Management Design	4/12/26	4/6/26
Implementation			
	PRD Finalization, AI-Assisted Prototype & System Demonstration	4/19/26	4/19/26
	Project Submission	4/26/26	